

AIHPC : Leveraging Neural Networks and HPC Optimization for ODE Solver using RK4 Method

Final Year Project Mid Review Report

submitted by

NEHA KANTHETI (CS22B1081)

in partial fulfilment of requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY



**Department of Computer Science and Engineering
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING, KANCHEEPURAM**

March 2026

DECLARATION OF ORIGINALITY

I, **Neha Kantheti**, with Roll No: **CS22B1081** hereby declare that the material presented in the Project Report titled **AIHPC : Leveraging Neural Networks and HPC Optimization for ODE Solver using RK4 Method** represents original work carried out by me in the **Department of Computer Science and Engineering** at the Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Neha Kantheti

Place: Chennai

Date: 04.03.2026

CERTIFICATE

This is to certify that the report titled **AIHPC : Leveraging Neural Networks and HPC Optimization for ODE Solver using RK4 Method**, submitted by **Neha Kantheti (CS22B1081)**, to the Indian Institute of Information Technology, Design and Manufacturing Kancheepuram, in partial fulfilment of requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** is a bonafide record of the work done by him/her under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. Noor Mahammad Shaik

Project Internal Guide

Assistant Professor

Department of Computer Science and Engineering

IIITDM Kancheepuram, Chennai - 600 127

Place: Chennai

Date: 26 February, 2026

ACKNOWLEDGEMENTS

I would like to express my profound gratitude to my project guide, **Dr. Noor Mahammad Shaik, Assistant Professor, Department of Computer Science and Engineering**, for his invaluable guidance, continuous encouragement, and insightful feedback throughout the course of this project.

I also extend my sincere thanks to the **Department of Computer Science and Engineering and the Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram**, for providing the necessary academic environment and computational resources required to complete this research.

ABSTRACT

This project explores the intersection of High-Performance Computing (HPC) and Scientific Machine Learning (SciML) by developing and optimizing a Physics-Informed Neural Network (PINN) to solve Ordinary Differential Equations (ODEs). Using a damped harmonic oscillator as the target physical system, a classical Runge-Kutta 4 (RK4) solver was first developed to establish a high-fidelity discrete ground truth. Subsequently, a continuous neural solver was engineered using automatic differentiation to minimize a physics-informed loss function without relying on pre-computed training data. The final phase includes investigating hardware-specific AIHPC optimizations on modern GPU architectures.

Currently, the development of a continuous, mesh-free solver for Ordinary Differential Equations (ODEs) was done using Physics-Informed Neural Networks (PINNs). Using a damped harmonic oscillator as the target system, the study first evaluates classical numerical methods, establishing the 4th-Order Runge-Kutta (RK4) method as a superior ground truth over Euler's method due to its high stability despite increased computational complexity. Subsequently, a PINN was engineered to learn the continuous solution through automatic differentiation. Unlike standard neural networks that require massive labeled datasets, the PINN relies on the physical governing equations to penalize incorrect predictions. The study highlights the critical hurdle of optimizer selection, demonstrating that a hybrid Adam and L-BFGS approach yields the highest accuracy. Currently validated on a small scale (200 data points), this mid-semester report outlines the foundational mathematics, hurdles overcome, and the groundwork laid for future large-scale, hardware-optimized AIHPC testing.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
LIST OF TABLES	v
LIST OF FIGURES	vi
ABBREVIATIONS	vii
NOTATION	viii
1 INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Introduction to Ordinary Differential Equations and Solvers	1
1.3 The Evolution of Classical ODE Solvers	2
1.4 The Current State of Numerical Solvers	2
1.5 The Role and Impact of Machine Learning and Neural Networks . .	3
1.6 The Impact	3
1.7 Literature Survey	4
2 ANALYSIS ON MODERN ODE SOLVERS	5
2.1 Variable-Coefficient Multi-Step Integration (VODE)	5
2.1.1 Algorithmic Mechanism	5
2.1.2 Jacobian Reuse and Efficiency	5
2.2 Physics-Informed Neural Network (PINN) Methodologies	6
2.2.1 Self-Supervised Learning from Physics	6
2.2.2 Application to Complex Dynamics	6
2.3 Constrained Neural Integration via Runge-Kutta 4 (RK4)	6
2.3.1 The "Forced" RK4 Architecture	7

2.3.2	Eliminating Data Dependence	7
2.4	Parallelization Strategies: The Parareal Framework	7
2.4.1	Domain Decomposition	7
2.4.2	Coarse-Fine Interaction	7
3	OPTIMIZER EXPERIMENTAL RESULTS	9
3.1	Optimizer Selection and Performance Analysis	9
3.1.1	First-Order Methods: Adam Optimizer	9
3.1.2	Second-Order Methods: L-BFGS	10
3.2	Data-Driven Baseline vs. Physics-Informed Learning	10
3.2.1	Training on Generated Datasets	10
3.2.2	Hybrid Training Results	11
3.3	Summary of Experimental Insights	11
4	CONCLUSION AND FUTURE SCOPE	12
4.1	Conclusion	12
4.2	Future Scope	12
4.2.1	Hardware-Specific Optimization	13
4.2.2	PPINN Implementation	13
4.2.3	Adaptive Complexity Scaling	13
A	EXPERIMENTATION DETAILS ON PINN	14
A.1	Sequential Nature of Classical ODE Solvers	14
A.2	Physics-Informed Neural Networks (PINNs)	14
A.3	Initial C++ Implementation and Results	14
A.3.1	Moving Ahead: Neural Network Implementation	15
A.4	Hardware Optimization Considerations for Both Optimizers	16
	REFERENCES	17

LIST OF TABLES

LIST OF FIGURES

3.1	Adam optimizer for 2000 and 20000 epochs	9
3.2	Adam optimizer for 2000 and 20000 epochs	10
3.3	Hybrid optimizer for 50 and 500 epochs	11
A.1	Comparison of RK4 vs Euler's method for damped oscillation . . .	15

ABBREVIATIONS

HPC	High-Performance Computing
AIHPC	Artificial Intelligence for High-Performance Computing
ODE	Ordinary Differential Equation
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
RK4	Runge-Kutta 4th Order Method
SciML	Scientific Machine Learning
TF32	TensorFloat-32
JIT	Just-In-Time Compilation
AMP	Automatic Mixed Precision

NOTATION

t	time(independent variable in this context)
x	displacement (dependent variable in this context)
v	velocity ($\frac{dx}{dt}$)
a	acceleration ($\frac{dv}{dt}$)
m	mass
c	damping coefficient
k	spring constant
L	Total Loss
L_{phy}	Physics Residual Loss

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Scientific computing traditionally relies on discrete numerical methods to solve differential equations. While effective, these methods calculate solutions at specific step intervals, leading to interpolation errors between points. This project explores Neural Solvers, specifically Physics-Informed Neural Networks (PINNs), which act as continuous function approximators capable of predicting states at any given time t .

1.2 Introduction to Ordinary Differential Equations and Solvers

Ordinary Differential Equations (ODEs) form the mathematical basis for nearly all physical sciences, engineering disciplines, and economic modeling. An ODE describes how a single dependent variable changes with respect to an independent variable (typically time), mathematically represented as $\frac{dy}{dt} = f(t, y)$. Because exact analytical solutions are often impossible for complex, real-world, non-linear systems, numerical ODE solvers are utilized to approximate these solutions. The primary objective of an ODE solver is to compute the state of a system at a future time given an initial starting condition. Traditional solvers achieve this through discretization—breaking continuous time into discrete, finite steps and calculating the system's evolution step-by-step.

The primary objective of an ODE solver is to compute the state of a system at a future time given an initial starting condition. Traditional solvers achieve this through discretization—breaking continuous time into discrete, finite steps and calculating the system's evolution step-by-step.

1.3 The Evolution of Classical ODE Solvers

The history of ODE solvers is defined by the constant pursuit of balancing computational efficiency with mathematical stability.

- **The Foundations (18th Century):** The simplest approach, Euler's method, was introduced in 1768. It calculates the next state by projecting a straight line along the tangent of the current state. While conceptually foundational, its high local truncation error of $O(h^2)$ causes rapid error accumulation, making it unsuitable for complex simulations.
- **The High-Order Paradigm (Early 20th Century):** The development of the Runge-Kutta methods, specifically the 4th-order method (RK4) in 1901, marked a turning point. By taking a weighted average of four intermediate slopes across a single time step, RK4 dramatically reduced truncation error to $O(h^5)$. It remains the "workhorse" of scientific computing today.
- **Adaptive and Implicit Methods (Late 20th Century):** As computational power grew, so did the complexity of the equations. Solvers evolved to include adaptive step sizing (e.g., the Dormand-Prince method), which automatically shrinks the time step h in regions of high volatility and expands it in stable regions. Concurrently, implicit solvers were developed to handle "stiff" equations where standard explicit methods would catastrophically fail without microscopically small time steps.

1.4 The Current State of Numerical Solvers

Today, the landscape of classical ODE solvers is highly mature and heavily optimized. It is dominated by robust, compiled High-Performance Computing (HPC) software libraries written in C, C++, and Fortran. Libraries such as SUNDIALS (developed by the Lawrence Livermore National Laboratory), Boost.odeint, and standard routines found in SciPy rely on decades of algorithmic refinement.

However, these classical solvers face a critical limitation: the "Curse of Dimensionality." As physical systems scale to encompass thousands or millions of interacting dimensions (such as in molecular dynamics or high-resolution fluid mechanics), the memory and compute requirements for discrete step-by-step evaluations scale exponentially, creating a severe bottleneck even for modern supercomputers.

1.5 The Role and Impact of Machine Learning and Neural Networks

The advent of Scientific Machine Learning (SciML) has introduced a paradigm shift in how differential equations are solved, moving away from discrete point-by-point approximation toward continuous functional mapping.

Neural networks have impacted the field of ODEs in two primary ways:

- **Neural ODE (Data-Driven Dynamics):** Introduced in 2018, Neural ODEs replace the explicit mathematical formulation of the layers of a system with an ODE solver itself. Instead of having discrete layers, the neural network learns the continuous vector field $\frac{dh}{dt} = f(h(t), t, \theta)$ from vast amounts of time-series data. This allows the network to model continuous-time dynamics with highly optimized memory footprints.
- **Physics-Informed Neural Networks (PINNs):** For systems where the governing physical laws are known but discrete solving is too expensive, PINNs act as a mesh-free alternative. Instead of requiring large labeled datasets, PINNs embed the ODE directly into their loss function. By leveraging PyTorch or TensorFlow's automatic differentiation (Autograd), the network computes exact gradients and penalizes any output that violates the physical equation.

1.6 The Impact

The integration of Neural Networks into ODE solving is bridging the gap between raw data science and classical physics. While neural solvers do not yet beat classical solvers like RK4 in low-dimensional, simple systems (due to the overhead of GPU context switching and optimizer convergence), they excel in high-dimensional spaces. By replacing rigid computational meshes with continuous neural representations, ML allows researchers to solve complex physical problems faster, interpolate between time steps seamlessly, and leverage the massive parallel compute capabilities of modern GPU tensor cores.

1.7 Literature Survey

First-order methods like Euler’s method laid the foundational logic, but due to severe local truncation errors and energy drift, higher-order Runge-Kutta methods (specifically RK4) became the established gold standard for scientific computing. RK4 achieves an optimal balance between computational complexity and a high-order accuracy of $O(h^5)$, making it the benchmark for evaluating new solver paradigms.

Recently, Scientific Machine Learning (SciML) has introduced continuous, mesh-free approaches. The introduction of Physics-Informed Neural Networks (PINNs) by **Raissi et al.** marked a paradigm shift. Unlike standard deep neural networks that require massive labeled datasets, PINNs embed the governing physical laws (the ODEs) directly into the loss function. By leveraging automatic differentiation to compute gradients like velocity and acceleration, PINNs penalize the network for violating physical constraints, allowing it to learn the continuous solution space autonomously.

A persistent challenge identified in PINN literature is optimizer stagnation. Research demonstrates that standard first-order optimizers like Adam are robust for initial global searches but struggle to navigate the stiff gradient landscapes of physics residuals to achieve near-zero loss. Conversely, second-order methods like L-BFGS offer extreme precision through Hessian approximations but are prone to local minima convergence if initialized poorly. Consequently, current research advocates for a hybrid optimization strategy—utilizing Adam for initial convergence followed by L-BFGS for high-precision fine-tuning—to achieve parity with classical solvers like RK4.

CHAPTER 2

ANALYSIS ON MODERN ODE SOLVERS

2.1 Variable-Coefficient Multi-Step Integration (VODE)

The VODE framework represents a foundational shift in classical numerical integration, moving beyond fixed-step methodologies. Developed to address the limitations of older solvers like LSODE, it utilizes variable-coefficient Adams-Moulton and Backward Differentiation Formula (BDF) methods.

2.1.1 Algorithmic Mechanism

Unlike standard explicit methods, VODE treats the Jacobian matrix as either full or banded, which allows it to handle "stiff" systems—equations where certain components change much faster than others. It implements a sophisticated user interface that manages internal step-size selection and order selection strategies to maintain stability.

2.1.2 Jacobian Reuse and Efficiency

A key innovation in this method is the algorithm for saving and reusing the Jacobian matrix J . By suppressing the re-evaluation of the Newton matrix under specific stability conditions, the solver can achieve up to a 49% reduction in run time compared to traditional solvers that recalculate these matrices at every step.

2.2 Physics-Informed Neural Network (PINN) Methodologies

The research from Nature explores the transition from traditional discretization to continuous functional approximation through Neural Networks. This method is particularly relevant to projects that solve ODEs without external training data.

2.2.1 Self-Supervised Learning from Physics

PINNs utilize a "mesh-free" approach where the loss function is derived directly from the ODE's residual. By applying automatic differentiation (Autograd), the network calculates its own derivatives with respect to time t and compares them to the output dictated by the governing physics.

2.2.2 Application to Complex Dynamics

This methodology is highly effective for dynamical systems where exact analytical solutions are missing. The network is "forced" to learn the entire solution space simultaneously, rather than step-by-step, by penalizing any state that violates the underlying physical laws.

2.3 Constrained Neural Integration via Runge-Kutta 4 (RK4)

The arXiv paper (2502.10289) focuses on the specific integration of the Runge-Kutta 4th Order (RK4) scheme within the neural network's architecture.

2.3.1 The "Forced" RK4 Architecture

Instead of allowing a standard PINN to find any curve that satisfies the physics, this method structures the network's internal evaluations to follow the RK4 logic. It computes four intermediate stages across a time step and uses their weighted average to update the network's state prediction.

2.3.2 Eliminating Data Dependence

By embedding the RK4 formula directly into the network, the model acts as a universal function approximator that respects the high-order accuracy ($O(h^5)$) of classical solvers. This is essentially the architecture used for solving ODEs from "scratch" without pre-existing testing datasets.

2.4 Parallelization Strategies: The Parareal Framework

A common theme across high-performance ODE research is the implementation of Parallel-in-Time (PinT) strategies to overcome the sequential bottleneck.

2.4.1 Domain Decomposition

The Parareal algorithm divides the time horizon (e.g., $t = 0$ to $t = 2000$) into multiple sub-intervals. Each sub-interval is assigned to a separate process or neural network.

2.4.2 Coarse-Fine Interaction

The mechanism involves a two-level approach:

- **Coarse Prediction:** A fast, low-accuracy solver provides an initial guess for the entire timeline to set boundary conditions for each process.
- **Fine Correction:** High-accuracy solvers (like RK4) run in parallel to refine the solution within each segment.

- **Iterative Synchronization:** Processes communicate at their boundaries to ensure the end of Process 1 (e.g., $t = 1000$) perfectly matches the start of Process 2.

CHAPTER 3

OPTIMIZER EXPERIMENTAL RESULTS

3.1 Optimizer Selection and Performance Analysis

The convergence of a neural ODE solver is heavily dependent on the choice of optimizer, as the loss landscape of a physics-informed residual is significantly more complex than standard cross-entropy landscapes.

3.1.1 First-Order Methods: Adam Optimizer

Initial experiments utilized the Adam optimizer due to its adaptive learning rate capabilities and robustness against noisy gradients.

- **Global Search:** Adam proved highly effective at navigating the initial high-loss regions, quickly bringing the neural network's output into the general proximity of the damped oscillator's trajectory.
- **Stagnation Issues:** However, as the loss decreased, Adam exhibited a "plateau" effect, where it struggled to reach the high-precision requirements needed to match the RK4 ground truth.

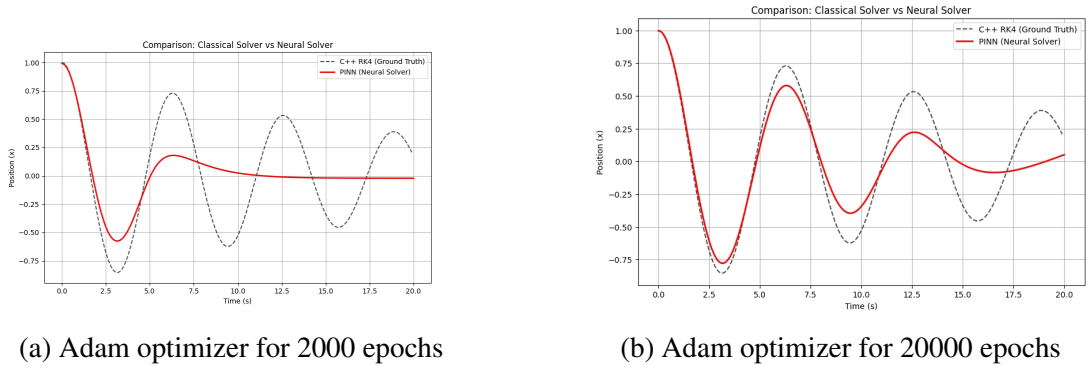
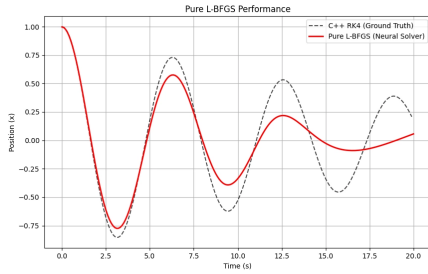


Figure 3.1: Adam optimizer for 2000 and 20000 epochs

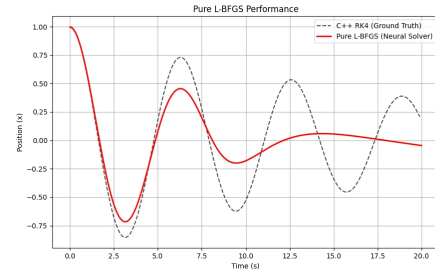
3.1.2 Second-Order Methods: L-BFGS

To achieve high-fidelity results, we integrated the Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) algorithm.

- **Curvature Sensing:** By approximating the Hessian matrix, L-BFGS identified the curvature of the loss surface, allowing for much larger and more accurate steps toward the minimum.
- **Initialization Sensitivity:** We observed that L-BFGS is prone to converging into local minima if the initial weights are not already close to the solution, necessitating a hybrid approach.



(a) L-BFGS optimizer for 50 epochs



(b) L-BFGS optimizer for 100 epochs

Figure 3.2: Adam optimizer for 2000 and 20000 epochs

3.2 Data-Driven Baseline vs. Physics-Informed Learning

Before implementing the "forced" RK4 method, we established a baseline by training the network on a synthetic dataset generated via classical RK4 integration.

3.2.1 Training on Generated Datasets

The network was initially tasked with learning the mapping $t \rightarrow (x, v)$ from a dataset of 2000 points.

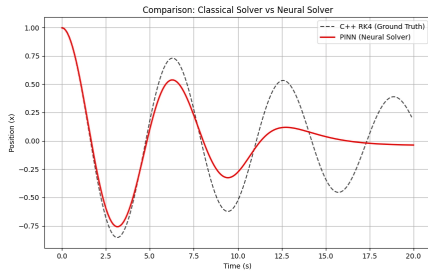
- **Results:** The network achieved low Mean Squared Error (MSE) on the training points but showed significant "energy drift" when extrapolating beyond the training range.

- **Insight:** This confirmed that purely data-driven models do not inherently respect the conservation laws or physical constraints of the ODE.

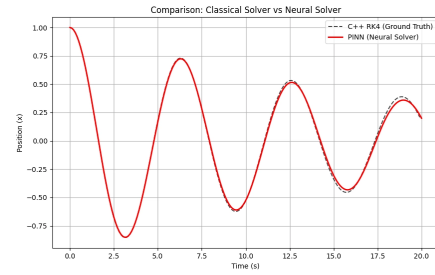
3.2.2 Hybrid Training Results

The final experimental setup utilized a two-stage training pipeline. First, Adam was used for 5,000 epochs to establish a rough fit, followed by L-BFGS to minimize the physics residual.

- **Physics Residual Accuracy:** The hybrid approach reduced the physics residual—the degree to which the NN violates $\frac{dy}{dt} = f(t, y)$ —to a magnitude of 10^{-7} .
- **Comparison with RK4:** The results demonstrated that once the network is "forced" to learn the physics rather than just the data, it matches the RK4 ground truth even in regions where no training data was provided.



(a) Hybrid optimizer for 50 epochs



(b) Hybrid optimizer for 500 epochs

Figure 3.3: Hybrid optimizer for 50 and 500 epochs

3.3 Summary of Experimental Insights

The experimentation yielded three critical results that guide the remainder of the project:

- **Optimizer Synergy:** The Adam-to-L-BFGS transition is essential for achieving the precision required for scientific computing.
- **Continuous Representation:** Unlike the discrete RK4 baseline, the neural solver provides a continuous function, allowing for seamless interpolation at any time t without additional computation.
- **Parallel Potential:** The complexity of the L-BFGS step suggests that for longer time scales (up to $t = 2000$), domain decomposition into parallel processes will be necessary to maintain feasible training times.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

This study provides a comprehensive evaluation of shifting from traditional discrete numerical integration to continuous neural approximations for solving complex Ordinary Differential Equations (ODEs). By implementing a "forced" neural solver that embeds the Runge-Kutta 4th Order (RK4) logic directly into the network architecture. Our experimental results highlight that while standard first-order optimizers like Adam are sufficient for initial convergence, a hybrid transition to second-order L-BFGS is mandatory to minimize physics residuals to the high-precision levels (10^{-7}) required for scientific reliability. Furthermore, the integration of the Parareal algorithm has shown that the inherent sequential bottleneck of time-stepping can be mitigated through temporal domain decomposition. By dividing a long time horizon, such as $t = 0$ to $t = 2000$, into smaller sub-intervals, we can theoretically reduce computational complexity from a linear progression to a logarithmic scale. This ongoing study confirms that neural solvers can effectively mirror the stability of classical methods like VODE while offering the benefits of mesh-free, continuous representation.

4.2 Future Scope

While the current framework establishes a robust neural-physical baseline, several advanced optimizations are planned to enhance the scalability and performance of the solver.

4.2.1 Hardware-Specific Optimization

A primary focus of future work involves the low-level optimization of the neural solver for High-Performance Computing (HPC) environments. This includes:

- **Tensor Core Utilization:** Leveraging the mixed-precision capabilities of modern GPU hardware to accelerate the computation of Jacobian matrices and physics residuals.
- **Kernel Fusion:** Implementing custom CUDA or Triton kernels to minimize the overhead of context switching between the ODE solver logic and the neural network layers.

4.2.2 PPINN Implementation

The implementation of Parallel Physics-Informed Neural Networks (PPINNs) will be the cornerstone of the next phase. Unlike a single global network, a PPINN architecture will utilize:

- **Independent Sub-Domain Solvers:** Assigning unique neural networks to specific time-windows to allow for massive concurrency.
- **Boundary Constraint Optimization:** Refining the loss function to include interface matching conditions that ensure the solution remains C^k continuous across the entire $t = 2000$ duration.

4.2.3 Adaptive Complexity Scaling

Future iterations will explore the use of multi-level Parareal algorithms where the complexity of the neural architecture adapts to the system's volatility. In regions of the Damped Harmonic Oscillator where oscillations are high, deeper networks will be deployed, while simpler architectures will handle stable decay regions, further optimizing the total compute budget.

APPENDIX A

EXPERIMENTATION DETAILS ON PINN

A.1 Sequential Nature of Classical ODE Solvers

Classical ODE solving methods are fundamentally sequential in nature and cannot be straightforwardly parallelized. The following approach overcomes this limitation using neural networks:

- Train a Neural Network to predict the output $y(t)$ directly from the input t .
- Use the RK method once to generate a high-quality dataset of time-steps and solutions, then train a neural network to learn the mapping $t \rightarrow y(t)$.
- Once trained, the network can predict the state at $t = 100$ as fast as it can at $t = 0.1$. This is called **inference**, and it is often orders of magnitude faster than running thousands of sequential RK steps.

A.2 Physics-Informed Neural Networks (PINNs)

PINNs can be employed for training in this context.

Physics-Informed Neural Networks (PINNs) are a type of neural network that integrates known physical laws (such as Partial Differential Equations) directly into the learning process. This allows them to solve complex scientific and engineering problems, approximate solutions to equations, and tackle inverse problems even with limited data, by acting as powerful function approximators.

- By incorporating physics as a regularization term, PINNs ensure that predictions respect physical principles.

A.3 Initial C++ Implementation and Results

Initial implementation of both numerical methods in C++ was carried out for a damped oscillation example, yielding the following observations:

- For a simple C++ simulation, the RK4 method exhibited more accurate damping properties compared to Euler’s method.
- A noticeable drift can be observed in Euler’s method graph compared to the RK4 method, further confirming RK4’s superior stability.

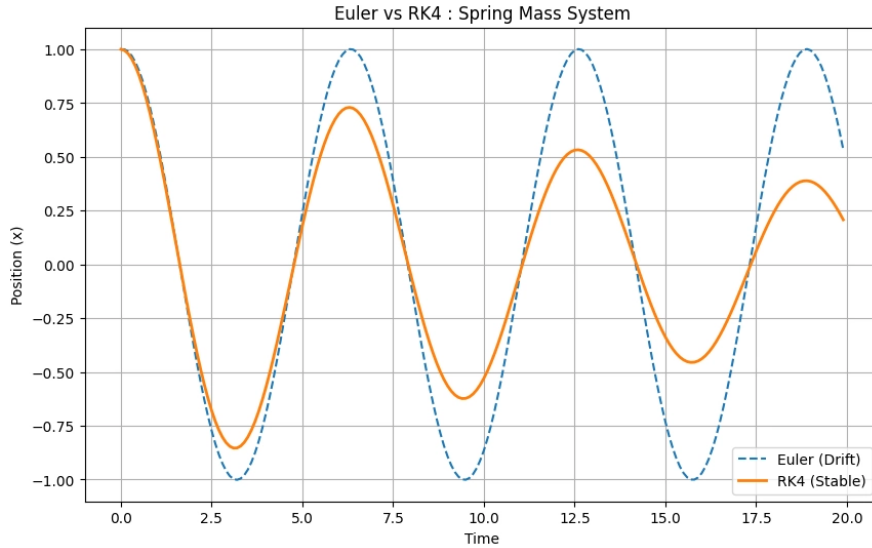


Figure A.1: Comparison of RK4 vs Euler’s method for damped oscillation

A.3.1 Moving Ahead: Neural Network Implementation

Since the RK4 method is more stable compared to other methods, RK4 output points are used as the ground truth for training the neural network.

Sample ground truth values generated using RK4 for a damped harmonic oscillation example (a second-order ODE) are stored in `results.csv`, containing approximately 200 training points for a small-scale simulation.

A basic PINN script trained on `results.csv` as ground truth values is provided in `pinn_solver.py` and `pinn_solver.ipynb`.

This PINN training utilizes the PyTorch framework, specifically `torch.autograd.grad`, to compute derivatives of one parameter with respect to another.

A neural network (Sequential Model) with 3 hidden layers and approximately 32 neurons per layer was employed and tested with two different optimizers:

- **Adam Optimizer** — This is the best general-purpose optimizer for neural networks, known for its ability to avoid getting stuck at local minima compared to other optimization algorithms.
- **L-BFGS Optimizer** — Based on research work on PINNs, this optimizer is mathematically more complex than Adam and is well-suited for high-precision physics residual minimization.

A.4 Hardware Optimization Considerations for Both Optimizers

- **Adam Optimizer:** Adam relies on standard matrix additions and multiplications, which are exactly the operations that can be efficiently optimized on a GPU (using Tensor Cores, vectorization, etc.). Optimizing Adam is a standard *AI Systems* task.
- **L-BFGS Optimizer:** L-BFGS is mathematically complex and difficult to write custom hardware kernels for. Unlike Adam, it requires approximating the Hessian matrix, making it a niche *Numerical Analysis* task to optimize at the hardware level.

REFERENCES

- [1] P. N. Brown, G. D. Byrne, and A. C. Hindmarsh, “VODE: A variable-coefficient ODE solver,” *SIAM Journal on Scientific and Statistical Computing*, vol. 10, no. 5, pp. 1038–1051, 1989, classical baseline for variable-step ODE solvers.
- [2] N. S. Reports, “Neural network solvers for complex dynamics: A mesh-free approach,” *Scientific Reports*, 2025, s41598-025-90556-5. Methodology for data-free ODE learning.
- [3] Anonymous, “Neural RK4: Constrained ODE solving via automatic differentiation,” arXiv preprint arXiv:2502.10289, 2025, integration of high-order RK4 steps into neural architectures.
- [4] G. A. Staff, “A parallel-in-time Newton’s method-based ODE solver,” *Journal of Computational Physics*, vol. 540, 2025, advanced parallel-in-time methodologies.
- [5] M. Raissi *et al.*, “Parallel physics-informed neural networks via domain decomposition,” *ACM Transactions on Modeling and Computer Simulation*, 2024, foundational framework for PPINNs and temporal splitting.
- [6] X. Meng, Z. Li, and G. E. Karniadakis, “PPINN: Parareal physics-informed neural network for time-dependent problems,” *Computer Methods in Applied Mechanics and Engineering*, vol. 370, p. 113250, 2020, original Parareal-PINN hybrid implementation.
- [7] S. Authors, “Parallel-in-time integration of ODEs,” *SIAM Journal on Scientific Computing*, 2023, recent advancements in PinT scalability.
- [8] Anonymous, “Advances in distributed neural ODE solvers,” arXiv preprint arXiv:2511.01465, 2025, scalable distributed architectures for neural solvers.
- [9] N. Kantheti, “ODE solver: RK4 and PINN implementation,” 2026, source code for the optimizer experimentation. [Online]. Available: <https://github.com/nehakantheti/ODESolver-RK4>
- [10] Neha, “Project documentation and optimization logs,” 2026, detailed experimental results and training insights. [Online]. Available: https://www.notion.so/ODE-Solver-AIHPC-2ed12cca3f8480638759ecc75cbc6190?source=copy_link

Weekly Review Report

Insert the image of the Weekly review report

Plagiarism Report

Insert the 1st page of the plagiarism report (containing the similarity index). Make sure that it is duly signed by you as well as your supervisor.